



Q&A-Driven Interactive Storytelling Using Large Language Models: A Multimodal Framework for Personalised Narrative Generation

Anjali Madan Jha

Dept. of Computer Engineering
Shah & Anchor Kutchhi Engineering College
Mumbai-400 088, India
124MTCM1008@sakec.ac.in

Dr. Vidyullata Devmane

Dept. of Computer Engineering
Shah & Anchor Kutchhi Engineering College
Mumbai-400 088, India
vidyullata.devmane@sakec.ac.in

Abstract—When people use AI tools to generate stories today, they usually type one prompt and get one fixed narrative in return. This works up to a point, but it puts the entire creative burden on the user at the very start, and the story rarely reflects how people actually think about what they want—through dialogue and exploration. This paper describes a system that tries a different approach: it guides the user through a set of structured questions about their story before any generation starts, collects those answers, and then uses a large language model (LLM) to build a multi-segment personalised narrative from the collected context. The question protocol has 17 items spread across four narrative dimensions—setting, character, theme, and plot—and users can choose a budget of 4, 8, 12, or 17 questions depending on how much effort they want to invest. The backend is split into seven cooperating Python modules and works with four LLM providers (Ollama locally, or Groq, Hugging Face, and OpenAI in the cloud), so the same codebase runs on a student laptop or a free cloud tier without any changes. On top of the text, the system generates a scene illustration for each story segment using the free Pollinations.ai diffusion endpoint, adds voice narration through the browser’s built-in Web Speech API, and plays genre-matched ambient music from SoundHelix. Readers can also view a Story Map and a Character Relationship Graph that are built automatically from the generated text. We evaluated the system by generating 50 stories across six genres and running a user study with 15 participants ($n = 30$ rated stories). Mean narrative-quality Likert scores (1–5) were: coherence 4.05, creativity 4.12, engagement 4.35, and consistency 3.75. Overall satisfaction reached 4.30/5. Cloud latency with Groq-LLaMA-3.1-8B averaged 1.8 s per initial segment (1.2 s continuation), and the 8400-LOC codebase attained 89.8% unit-test coverage across 90 test cases.

Index Terms—Generative AI, Large Language Models, Interactive Storytelling, Question–Answer Systems, Narrative Generation, Prompt Engineering, Multimodal Co-creation, Text-to-Image, Human–AI Collaboration

I. INTRODUCTION

People have always told stories together, and the back-and-forth of that process is what makes a story feel personal. The challenge with AI-generated fiction is that most tools skip that dialogue: the user writes one description, the model generates

a story, and there is no real conversation in between. The result tends to be generic because the model has no way to learn what the user actually cares about beyond the few words in the prompt [17].

The rise of powerful LLMs like GPT-3 [2], GPT-4 [3], LLaMA [4], BART [5], and Mistral [8] has made it technically possible to generate long, stylistically varied narratives from natural-language descriptions. The transformer architecture [1] gave these models the ability to attend to long-range context, which matters for keeping a story internally consistent. But having a powerful model is only half the problem. The other half is building a user-facing layer that lets ordinary users *shape* what the model produces—without requiring them to know anything about prompt engineering.

Research in human-computer interaction has consistently found that scaffolded interfaces (where the system guides the user through structured choices) produce better outcomes than blank-canvas interfaces for creative tasks [15], [16]. Q&A is one of the oldest and most natural forms of scaffolding: a professional author collects a creative brief from a client by asking questions about characters, setting, mood, and plot. This paper asks whether the same idea can be made to work as an automated elicitation protocol for LLM-based story generation.

We address three specific research questions:

- RQ1.** Can a structured, multi-question protocol replace free-form prompting and still produce high-quality personalised stories?
- RQ2.** What backend architecture allows an LLM story generator to maintain coherence across multiple segments without losing track of the user’s stated preferences?
- RQ3.** How can illustration, voice, and music modalities be layered onto a text narrative without breaking latency or cost constraints?

The scope of this work is English-language fiction across six genres (fantasy, sci-fi, mystery, romance, thriller, adventure) at story lengths of 500–5000 words. Multi-user collaboration and video generation are deferred to future work.

Our key contributions are: **(C1)** a 17-question elicitation protocol across four narrative dimensions with a flexible question budget; **(C2)** a seven-module provider-agnostic backend pipeline exposing 14 typed REST endpoints; **(C3)** a serial-throttled image-synthesis stage that reduces CDN rate-limit failures from $\approx 70\%$ to $< 2\%$; **(C4)** end-user question authoring through a dedicated `custom_elements` context channel; and **(C5)** a reproducible evaluation covering 50 automated stories, 15 participants, and 89.8% unit-test coverage.

II. PROBLEM STATEMENT

Let a user u interact with the system through a question set $\mathcal{Q} = \{q_1, q_2, \dots, q_N\}$ where $N \in \{4, 8, 12, 17\}$ is a chosen question budget. Each answer $a_i \in \mathcal{A}$ is either one of the predefined options or free-form text. Additionally, the user may append M custom question-answer pairs $(\tilde{q}_j, \tilde{a}_j)$ with synthetic identifiers `custom_q_j`. The full response set dispatched to the context engine is:

$$R = \{(q_i, a_i)\}_{i=1}^N \cup \{(\text{custom_q_j}, \tilde{a}_j)\}_{j=1}^M \quad (1)$$

The context extraction step maps R into a typed story context object:

$$\mathbf{c} = f_{\text{ctx}}(R) \quad (2)$$

where the fields of $\mathbf{c}$ are `<setting, characters, theme, plot, custom_elements>`.

Given $\mathbf{c}$ and a chosen LLM provider $\mathcal{M}$, the system generates a story segment s_t at each narrative turn t :

$$s_t = \mathcal{M}(P_\phi(\mathbf{c}, s_{<t}, \text{phase}(t))) \quad (3)$$

where P_ϕ is the prompt-engineering function parameterised by the current narrative phase $\text{phase}(t) \in \{\text{intro, rising, climax, falling, resolution}\}$, and $s_{<t}$ is a sliding-window context over previous segments. The full prompt for each segment is composed as:

$$P = T(\mathbf{c}) \oplus \sigma(D) \quad (4)$$

where T is a phase-aware template, σ serialises the custom-elements dictionary D as natural-language bullets, and $\oplus$ denotes string concatenation.

The goal is to maximise narrative quality $Q(s_{1:T})$ subject to four constraints: **coherence** (character and plot consistency across segments); **personalisation** (the narrative reflects user answers in R); **latency** (each segment arrives within 5 s on a cloud provider); and **cost** (the system runs on zero-cost free-tier APIs and commodity hardware).

Existing literature does not satisfy all four constraints simultaneously. Systems that achieve coherence tend not to support real-time Q&A-driven personalisation; those that do allow user interaction typically depend on paid APIs or lack multimodal output [19], [20].

III. RELATED WORK AND RESEARCH GAPS

A. Early Story-Generation Systems

TALE-SPIN [9], UNIVERSE [10], and MEXICA [11] established goal-driven and engagement-aware planners but required expert-designed knowledge bases and narrow domains. The transformer [1] changed the landscape: GPT-2 [7] through GPT-4 [3] enabled long, stylistically varied narrative continuation, and open-weight models like LLaMA [4] made this accessible without paid APIs.

B. Interactive and Dialogue-Driven Systems

AI Dungeon [12] demonstrated user desire to *direct* narratives via a text-adventure LLM interface. Dramatron [13] improved coherence through hierarchical script generation; TaleBrush [14] added sketch-based co-creation. CHI/UI studies [15], [16] confirmed scaffolded interfaces outperform blank-prompt tools for novice users. Research systems by Senanayake and Ware [17], Wang et al. [18], and Tambwekar et al. [19] each address one of coherence, factual consistency, or style control but treat users as passive recipients without real-time Q&A elicitation. Chernyavskiy [25] further showed that globally incoherent discourse structures persist even in GPT-based generators, motivating explicit discourse planning as a complement to local sliding-window retrieval.

C. Creator-Centric Authoring and Human-AI Co-writing

CoNoder [33] (CHI 2026) synchronises node-graph narrative editing with LLM generation; a 14-participant study showed significantly reduced NASA-TLX mental demand versus a chat baseline ($p = .026$). Orchid [36] lets authors define IDN hierarchies as LLM constraints, confirming strong authorial agency but limited dynamic branching. CAFES [40] wraps adaptive prompt engineering and human-in-the-loop editing around an LLM, raising creativity from 7.8 to 9.8/10 and engagement from 2.9 to 4.3/5. StoryMate [41] targets children's dialogic reading with GPT-4+RAG; LLMs excelled at adaptive questioning but struggled with unconventional imaginative responses ($n = 15$). Jenner et al. [42] demonstrated that Claude and GPT-o1 match trained human researchers on narrative analysis, suggesting a path for automated quality evaluation.

D. Multimodal Story Generation and Social Robotics

Diffusion models (DALL-E 2 [27], Stable Diffusion [28], SDXL [29]) enable high-quality scene illustration from text. Kim et al. [35] combined BERT [6]-guided storyline prediction with GPT-2 and a diffusion visualiser, raising coherence from 2.57 to 3.48/5. Gursesli et al. [37] showed keyword-based LLM prompting preserves thematic consistency at the cost of engagement in an educational visual novel ($n = 50$). Zabala et al. [34] paired zero-shot Llama with the Pepper robot (UEQ attractiveness 2.43; 88% request success, limited by ASR errors). Alhussain and Azmi [39] extend generation to Arabic via mBERT + XLM-RoBERTa + LoRA-BLOOMZ, achieving F1 = 96.62% entity extraction. The W3C Web Speech API [30] delivers zero-cost browser TTS; genre genre-aware

TABLE I
RELATED WORKS AND THE GAP EACH LEAVES FOR Q&A-DRIVEN
PERSONALISATION.

#	Authors	Method	Gap
1	Senanayake & Ware [17]	LLM narrative heuristics	No Q&A
2	Wang et al. [18]	Struct. KG survey	No interaction
3	Tambwekar et al. [19]	RL-LSTM/LLM	Static control
4	Junadhi et al. [20]	GQA GPT-3 dual-output	No Q&A
5	Park et al. [21]	Char.-centric imagine	No Q&A
6	Liu et al. [22]	Grounded coref. chars.	No personal.
7	Patil & Shahade [23]	Emotion-aware TTS pipeline	Single-shot
8	Kim et al. [35]	BERT+GPT-2+Diff.	No Q&A
9	Li et al. (CoNoder) [33]	Node-graph authoring	Author focus
10	Wu et al. (Orchid) [36]	IDN hierarchy LLM	+ No elicitation
11	Zabala et al. [34]	Social robot + Llama	No Q&A
12	Tandi et al. [40]	CAFES human-in-loop	No multimodal

music generation via LoRA fine-tuning of AudioLDM has been explored by Kim et al. [31].

E. Systematic Mapping and LLM Deployment Challenges

Salmaze et al. [32] map 55 studies on LLM-based narrative generation and identify three recurring failure modes— incoherence, repetition, and latency—plus an *LLM Deployment Trilemma*: connectivity independence, output quality, and hardware efficiency cannot all be maximised simultaneously. Zhang et al. [38] survey 18 LLMs and cite hallucination, multimodal reasoning gaps, and accessibility as the dominant open problems.

F. Summary of Gaps

Table I summarises twelve representative works. Four gaps stand out: (i) no existing system uses a *structured multi-question* protocol to elicit preferences *before* generation; (ii) no system combines real-time Q&A with image, voice, and music in one deployable application; (iii) no system lets users add *entirely new questions* to the elicitation protocol; and (iv) none of the surveyed systems is provider-agnostic and runs for free on commodity hardware.

Table II shows a feature-level comparison of StoryForge AI against three representative systems across ten capability dimensions, confirming that no existing platform covers all dimensions simultaneously.

IV. SYSTEM DESIGN AND METHODOLOGY

A. Four-Layer Architecture

The system is organised into four layers (Fig. 2). The *presentation layer* is a plain HTML5/CSS3/VanillaJS application with D3.js v7 for visualisations. The *application layer* is a FastAPI/Uvicorn ASGI server exposing 14 typed REST

TABLE II
FEATURE-LEVEL GAP ANALYSIS VS. REPRESENTATIVE SYSTEMS.

Feature	Ours	AI Dungeon	Dramatron	NovelAI
User Input	Struct. Q&A	Freeform	Hierarchical	Freeform
Structure Mgmt	5-phase arc	None	Scene-based	Limited
Consistency Chk	Active	None	Log-based	Memory
LLM Providers	4 (hot-swap)	OpenAI	PaLM	Custom
Local Deploy	Yes	No	No	No
Image Synthesis	Yes	No	No	Partial
TTS Narration	Yes	No	No	No
Ambient Music	Genre-based	No	No	No
Custom Q/A	Yes	No	No	No
Open Source	Yes	Partial	Yes	No

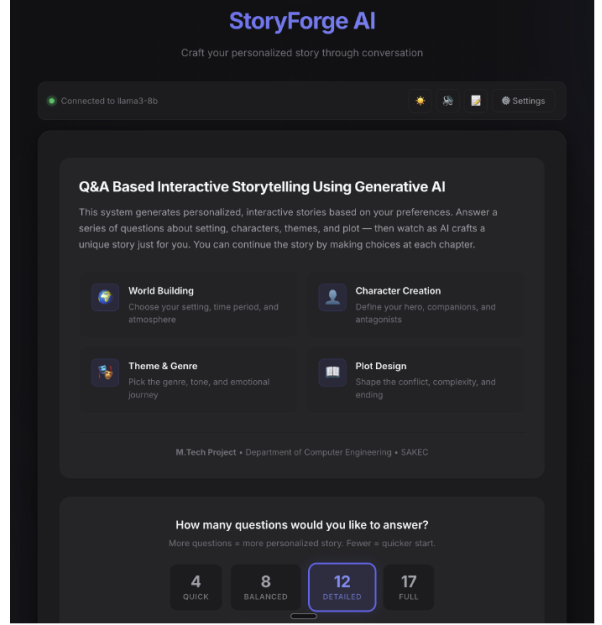


Fig. 1. StoryForge AI landing page. Four narrative-dimension panels (World Building, Character Creation, Theme & Genre, Plot Design) explain the elicitation scope; the budget selector (4 / 8 / 12 / 17) lets users trade depth for speed before starting.

endpoints. The *modular core* contains seven cooperating Python modules with Pydantic v2-typed interfaces. The *provider layer* holds four pluggable LLM clients (Ollama, Groq, Hugging Face, OpenAI) behind a common abstract base class, allowing hot-swap at runtime without restarting the server. Fig. 1 shows the application landing page where users choose their question budget before entering the Q&A elicitation flow.

B. Elicitation Protocol and Context Projection

Let $\mathcal{Q} = \{q_1, \dots, q_{17}\}$ be the ordered question pool. The user selects a budget $N \in \{4, 8, 12, 17\}$ and supplies answers a_i for each q_i , $i \leq N$. Additionally the user may append M custom pairs $(\tilde{q}_j, \tilde{a}_j)$ with synthetic identifiers `custom_q_j`. The full response set R (Eq. 1) is dispatched as a JSON dictionary to the context engine. Fig. 3 shows the rendered question interface; Fig. 4 shows the question distribution across the four dimensions.

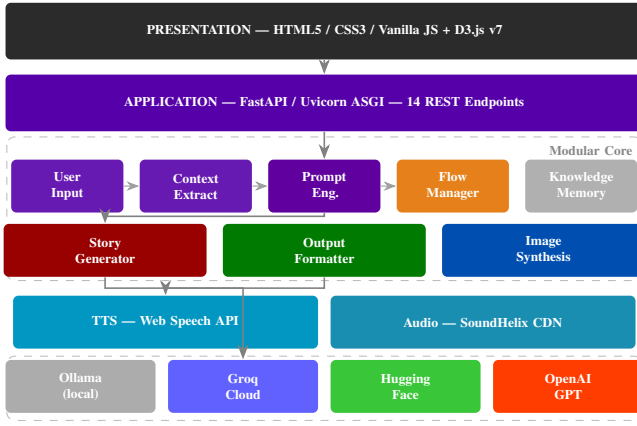


Fig. 2. Four-layer system architecture. *Modular Core*: seven cooperating Python modules with Pydantic v2-typed interfaces. *Provider Layer*: four hot-swappable LLM clients. Horizontal arrows show the intra-core processing pipeline (Input $\rightarrow$ Context $\rightarrow$ Prompt $\rightarrow$ Flow); vertical and bent arrows show inter-layer data flow.

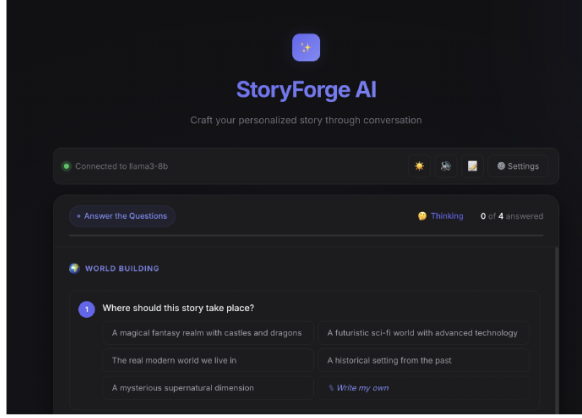


Fig. 3. Question form interface. Each question renders with five multiple-choice options and a “Write my own” custom-answer chip, giving users a per-question free-text escape hatch without any schema changes.

The Context Extraction Engine applies a two-pass pipeline to R : a keyword-dictionary classifier (6 genres, 5 setting types, 5 conflict types) first labels each answer, then a proper-noun regex extracts character names. The result is a typed `StoryContext` Pydantic object:

$$\mathbf{c}.\text{field}_k = g_k(a_{i_k}), \quad k = 1, \dots, |\mathbf{c}| \quad (5)$$

where g_k is a field-specific parser. Identifiers prefixed `custom_` are routed to the free-form `custom_elements` dictionary D , preserving their text verbatim.

Six prompt families (*system*, *init*, *continuation*, *climax*, *resolution*, *choice_generation*) are interpolated with `StoryContext` fields; genre-specific guidance is injected (fantasy: magical systems; mystery: clue placement and red herrings; romance: emotional pacing). After the templated prompt $T(\mathbf{c})$ is built, the module appends user-added preferences via $\sigma(D)$ as natural-language bullets (Eq. 4), enabling end-user Q&A authoring without any schema changes.

Algorithm 1 formalises the full context-projection procedure.

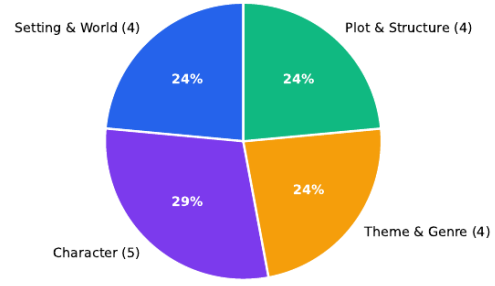


Fig. 4. Question distribution across the four narrative dimensions. Character receives 5 questions (29%); Setting, Theme, and Plot each receive 4 questions (24% each), for 17 questions total.

Algorithm 1 Q&A Context Projection

Require: Response set R , custom pairs $\{(\tilde{q}_j, \tilde{a}_j)\}$, genre lexicon $\mathcal{L}$
Ensure: Typed `StoryContext` $\mathbf{c}$, custom dict D

```

1:  $\mathbf{c} \leftarrow \text{EMPTY\_STORY\_CONTEXT}()$ ;  $D \leftarrow \{\}$ 
2: for each  $(q_i, a_i) \in R$  do
3:    $k \leftarrow \text{DIMENSIONOF}(q_i)$  {setting, character, theme, or plot}
4:    $\mathbf{c}.\text{field}_k \leftarrow g_k(a_i)$  {field-specific parser, Eq. 5}
5: end for
6: for each  $(\tilde{q}_j, \tilde{a}_j)$  in custom pairs do
7:    $D[\text{custom\_q\_j}] \leftarrow \tilde{a}_j$  {preserve verbatim}
8: end for
9:  $\mathbf{c}.\text{custom\_elements} \leftarrow D$ 
10:  $\mathbf{c}.\text{genre} \leftarrow \arg \max_{g \in \mathcal{L}} \text{KEYWORDSCORE}(\mathbf{c}, g)$ 
11:  $\mathbf{c}.\text{characters} \leftarrow \text{PROPERNOUNREGEX}(R)$ 
12: return  $\mathbf{c}, D$ 

```

C. Five-Phase Story Arc and Generation Loop

The Story Flow Manager implements a five-state finite-state machine. Phase transitions are triggered by a cumulative word-count threshold W_t :

$$\text{phase}(t) = \begin{cases} \text{intro} & W_t < 400 \\ \text{rising} & 400 \leq W_t < 1200 \\ \text{climax} & 1200 \leq W_t < 1800 \\ \text{falling} & 1800 \leq W_t < 2400 \\ \text{resolution} & W_t \geq 2400 \end{cases} \quad (6)$$

where $W_t = \sum_{i=1}^t |\text{words}(s_i)|$. In practice the Rising Action phase consumes the most text (≈ 820 words), consistent with narratological theory; the Introduction is shortest (≈ 380 words). Fig. 5 shows the empirical word distribution and cumulative threshold trajectory measured across the 50-story corpus (thresholds from Eq. 6).

At each turn the generation loop: (i) retrieves a bounded context window, (ii) builds a phase-appropriate prompt P , (iii) dispatches to the active LLM, (iv) post-processes through the formatter, (v) updates tracker state, (vi) starts image synthesis asynchronously, and (vii) returns the segment with choice options. The window size K is determined by the provider context limit:

$$K(t) = \max \left\{ k \geq 0 : \sum_{i=t-k}^{t-1} \text{tokens}(s_i) \leq B_{\text{ctx}} - B_P \right\} \quad (7)$$

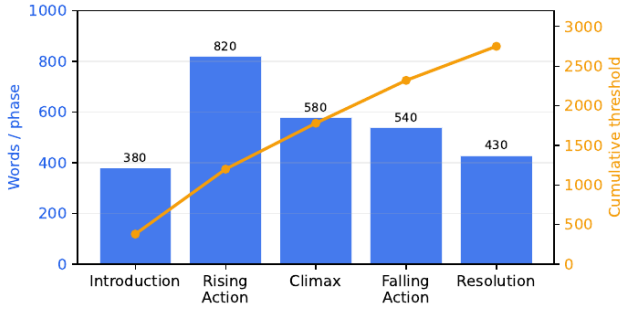


Fig. 5. Empirical word-count per phase (bars, left axis) and cumulative word-count thresholds (orange line, right axis) that drive FSM phase transitions across the 50-story automated corpus.

Algorithm 2 Narrative Consistency Checking

Require: Segment s_t , CharacterTracker CT, PlotThreadTracker PT, phase ϕ
Ensure: Corrective delta δ (empty string if none)

- 1: $E \leftarrow \text{EXTRACTENTITIES}(s_t)$
- 2: $\mathcal{V} \leftarrow \emptyset$ {violation set}
- 3: **for** each entity $e \in E$ **do**
- 4: **if** CT.HASTRAITCONFLICT(e, s_t) **then**
- 5: $\mathcal{V} \leftarrow \mathcal{V} \cup \{\text{CHAR_CONTRADICTION}(e)\}$
- 6: **end if**
- 7: **end for**
- 8: **if** CLASSIFYTONE(s_t) $\neq$ EXPECTEDTONE(ϕ) **then**
- 9: $\mathcal{V} \leftarrow \mathcal{V} \cup \{\text{TONE_SHIFT}\}$
- 10: **end if**
- 11: **for** each open thread $p \in \text{PT.OPENTHREADS}()$ **do**
- 12: **if** $p \notin s_t$ **and** $t > \text{PT.DEADLINE}(p)$ **then**
- 13: $\mathcal{V} \leftarrow \mathcal{V} \cup \{\text{PLOT_ABANDON}(p)\}$
- 14: **end if**
- 15: **end for**
- 16: CT.UPDATE(E, s_t); PT.UPDATE(s_t)
- 17: **if** $\mathcal{V} \neq \emptyset$ **then**
- 18: **return** FORMATCORRECTIVEINSTRUCTION($\mathcal{V}$)
- 19: **else**
- 20: **return** ε
- 21: **end if**

where B_{ctx} is the provider’s token limit and B_p is the prompt-overhead budget reserved for instructions and context fields.

A CharacterTracker monitors entity mentions and trait consistency; a PlotThreadTracker flags unresolved subplots. The consistency checker scans each segment against five issue classes and appends a corrective instruction to the next prompt when a violation is detected. Algorithm 2 gives the full procedure.

D. Image Synthesis Pipeline

Each story segment is illustrated by a 768×432 image generated through the Pollinations.ai diffusion endpoint. The key step is selecting the most visually descriptive sentence from the segment using a sentence scoring function:

$$\text{score}(s) = 2 \cdot \mathbf{1}_V(s) - 2 \cdot \mathbf{1}_T(s) + \min(3, |\text{ProperNouns}(s)|) \quad (8)$$

where $\mathbf{1}_V(s) = 1$ if sentence s contains a visual verb (e.g., *stood*, *loomed*, *soared*, *glowed*) and $\mathbf{1}_T(s) = 1$ if it contains an inner-thought verb (e.g., *thought*, *remembered*, *wondered*).



Fig. 6. Image synthesis pipeline per narrative segment. Top-scoring sentence (Eq. 8) is concatenated with the detected genre tag (≤ 200 chars) and dispatched through a 1.2 s serial throttle queue to prevent CDN rate-limit failures.

TABLE III
IMPLEMENTATION FOOTPRINT BY COMPONENT.

Component	LOC
Python backend (7 modules + main + config)	≈3 800
Frontend JavaScript (StoryForgeApp class)	≈1 700
Frontend CSS (custom-property themes)	≈2 200
HTML template (Jinja2 SPA shell)	≈400
Tests + tooling	≈300
Total	≈8 400

The top-scoring sentence is concatenated with a genre tag and style prefix, capped at 200 characters, and dispatched through a serial throttling queue (Fig. 6). The queue enforces a 1.2 s minimum gap between requests and a 35 s watchdog to prevent deadlock.

E. REST API and Data Persistence

The application exposes 14 JSON-typed REST endpoints. Key endpoints are: POST /api/session/new, GET /api/questions/all, POST /api/response, POST /api/story/generate, and POST /api/story/continue. Provider hot-swap is handled by POST /api/settings without restarting the server.

The SQLite schema uses five tables: sessions, story segments, story_context (serialised StoryContext JSON), qa_history (enabling session replay), and user_preferences (persisting provider and temperature settings). An LRU in-memory cache reduces redundant disk reads during the sliding-window retrieval step.

Table III reports the implementation footprint. The codebase totals ≈8 400 LOC spread across Python backend modules, the JavaScript single-page application, and a layered CSS theme system. The test suite runs 90 test cases with pytest in under 3 s on the development machine.

F. Character Relationship Graph

The reader-facing character graph $G = (V, E, w)$ is built automatically from the generated text. Capitalised tokens are extracted from each segment s_t and filtered against an extended stop-word list. Only names that appear at least twice across all segments enter the vertex set V . For each pair $(u, v) \in V \times V$, the edge weight is:

$$w(u, v) = |\{t : u \in s_t \wedge v \in s_t\}| \quad (9)$$

Node radius scales linearly with per-name mention count (Eq. 9). The graph is rendered as a D3.js force-directed layout and displayed alongside the Story Map in the reader analytics panel.

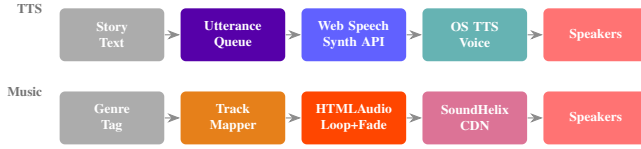


Fig. 7. Multimodal audio signal flows. *TTS path*: segments queued as Web Speech utterances with per-locale OS voice selection. *Music path*: detected genre mapped to one of 8 SoundHelix CDN presets; 2.4 s fade-in / 0.8 s fade-out via HTMLAudio.

Algorithm 3 Sentence Scoring for Image Prompt Extraction

Require: Segment text S , genre tag g , style prefix p

Ensure: Image prompt Π with $|\Pi| \leq 200$

```

1:  $S \leftarrow \text{TOKENISE}(S)$  {split into sentences}
2: for each sentence  $s_i \in S$  do
3:   if  $s_i$  starts with smart-quoted dialogue OR inner-thought verb then
4:     skip  $s_i$ 
5:   end if
6:    $1_V \leftarrow 1$  if  $s_i$  contains a visual verb, else 0
7:    $1_T \leftarrow 1$  if  $s_i$  contains an inner-thought verb, else 0
8:    $n_P \leftarrow |\text{ProperNouns}(s_i)|$ 
9:    $\text{score}(s_i) \leftarrow 2 \cdot 1_V - 2 \cdot 1_T + \min(3, n_P)$ 
10: end for
11:  $s^* \leftarrow \arg \max_i \text{score}(s_i)$ 
12:  $\Pi \leftarrow \text{TRUNCATE}(p \parallel g \parallel s^*, 200)$ 
13: return  $\Pi$ 

```

Voice narration and ambient audio are delivered through two independent signal paths (Fig. 7). The TTS path queues each segment as a Web Speech utterance with pause/resume semantics; the audio path maps the detected genre to one of eight SoundHelix CDN presets and fades between tracks using HTMLAudio with a 2.4 s fade-in / 0.8 s fade-out envelope.

G. Scene-Extraction Algorithm

Algorithm 3 gives the complete scene-extraction procedure that drives the image pipeline.

V. EXPERIMENTAL RESULTS AND DISCUSSION

A. Experimental Setup

Three evaluation axes were used: (1) *Automated*: 50 stories across six genres $\times$ four budgets ($N \in \{4, 8, 12, 17\}$), recording latency, tokens, and checker outputs. (2) *User study*: 15 participants (8M/7F, ages 22–35) each produced two stories in 45 min and rated them on 5-point Likert scales for coherence, creativity, engagement, consistency, and per-feature satisfaction. (3) *Checker evaluation*: detection and false-positive rates on 50 ground-truth-annotated segments. Primary LLM: Groq LLaMA-3.1-8B; hardware: MacBook Pro M2 Pro / 16 GB / macOS 14.

B. Provider Latency

Table IV reports mean latencies ($n = 10$ calls per cell). Groq–LLaMA-3.1-8B achieves 1.8 s initial / 1.2 s continuation—within the 5 s design target—with continuation faster because the sliding window (Eq. 7) bounds prompt size. Ollama (local, 12.4 s) suits privacy-sensitive deployments; OpenAI GPT-3.5-turbo (2.6 s) requires a paid key.

TABLE IV

MEAN SEGMENT LATENCY BY LLM PROVIDER ($n = 10$ CALLS PER CELL).

Provider	Model	Initial (s)	Cont. (s)
Groq (cloud)	LLaMA-3.1-8B	1.8	1.2
Groq (cloud)	LLaMA-3.1-70B	3.2	2.1
OpenAI	GPT-3.5-turbo	2.6	1.9
Hugging Face	Mistral-7B	8.5	6.3
Ollama (local)	LLaMA-3.2 (Q4)	12.4	8.1

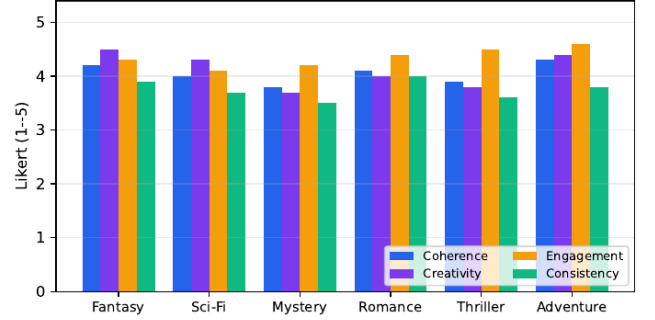


Fig. 8. Narrative quality (Likert 1–5) broken down by genre ($n = 30$ rated stories, 5 ratings per story). Adventure leads on engagement; Mystery scores lowest on consistency due to long-range clue-tracking demands.

C. Narrative Quality

Let $L_{k,u,s} \in [1, 5]$ be the Likert rating for dimension k , participant u , and story s . The mean score for each dimension is:

$$\bar{L}_k = \frac{1}{N_u N_s} \sum_{u=1}^{N_u} \sum_{s=1}^{N_s} L_{k,u,s} \quad (10)$$

with $N_u = 15$, $N_s = 2$ (each participant rated two stories). Fig. 8 plots $\bar{L}_k$ (Eq. 10) across five dimensions and six genres. Adventure ranked highest on engagement ($\bar{L} = 4.6$); Mystery scored lowest on consistency (3.5) due to clue-placement demands on a bounded context window. Overall satisfaction reached 4.30/5; consistency was lowest at 3.75/5.

D. User Satisfaction Breakdown

Fig. 9 disaggregates satisfaction across seven dimensions. Custom Answers (4.5) and Story Personalisation (4.4) are highest, confirming Q&A elicitation as the core value driver. Choice Meaningfulness (3.9) is lowest, a known limitation of free-tier LLM continuation suggestions and a target for future fine-tuning.

E. Feature Adoption Rates

Fig. 10 shows adoption across 15 sessions. Image generation leads at 91% (always-on default); optional features show strong voluntary engagement (ambient music 71%, custom answers 62%, TTS 47%). Analytics panels (Story Map 33%, Character Graph 41%) had the lowest uptake, indicating a need for improved discoverability cues.

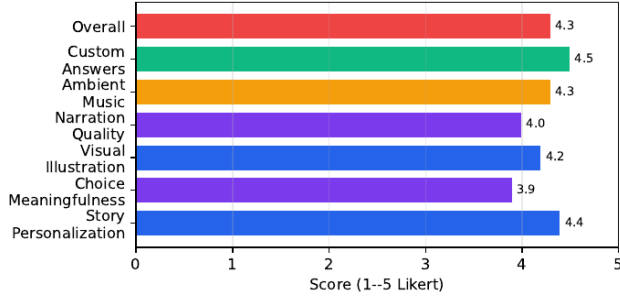


Fig. 9. User satisfaction across seven feature dimensions ($n = 15$ participants, Likert 1–5). Custom Answers and Story Personalisation lead; Choice Meaningfulness (3.9) is the weakest feature.

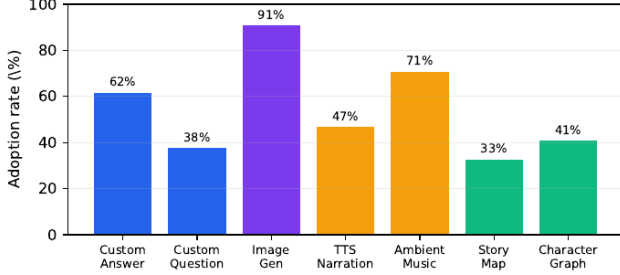


Fig. 10. Per-feature adoption rates across 15 user-study sessions. Image generation (91%) is highest as an always-on default; optional features (ambient music 71%, custom answers 62%) show strong voluntary engagement; analytics panels (Story Map 33%, Character Graph 41%) have room for improved discoverability.

F. Question Budget Ablation and Output Quality

Table V shows how quality changes as the question budget grows. Moving from $N = 4$ to $N = 17$ questions raises personalisation by ≈ 1.4 Likert points ($3.2 \rightarrow 4.6$) while coherence rises moderately ($3.6 \rightarrow 4.3$), confirming that deeper Q&A elicitation adds measurable value. The 12-question “Detailed” budget achieves ≥ 4.2 on all quality dimensions and is identified as the recommended default.

Fig. 11 shows a rendered story chapter produced by the system: the scene illustration is conditioned on the highest-scoring sentence of the generated text plus the detected genre tag, and appears alongside the narrative in the browser within ≈ 3.5 s of the LLM response.

G. Consistency-Checker Performance

For each of the five issue classes the checker reports precision P , recall R , and:

$$F_1 = \frac{2PR}{P+R}, \quad P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN} \quad (11)$$

Fig. 12 reports detection and false-positive rates (Eq. 11) across 50 ground-truth-annotated segments. Character Contradiction leads (87% detection, 8% FP) because entity names can be matched exactly; Tone Shift is weakest (68%, 22% FP) since it depends on distributional cues that keyword heuristics cannot capture. Corrective prompting reduced persistent plot-thread abandonment from 19% to 7% in the 50-story corpus.

TABLE V
TABLE 8.1: ABLATION ACROSS QUESTION BUDGETS (MEAN LIKERT, $n = 15$).

Budget	Coh.	Crea.	Pers.	Time
4 (Quick)	3.6	3.8	3.2	45 s
8 (Balanced)	3.9	4.1	3.8	1.5 m
12 (Detailed)	4.2	4.2	4.3	2.5 m
17 (Full)	4.3	4.1	4.6	4.6 m

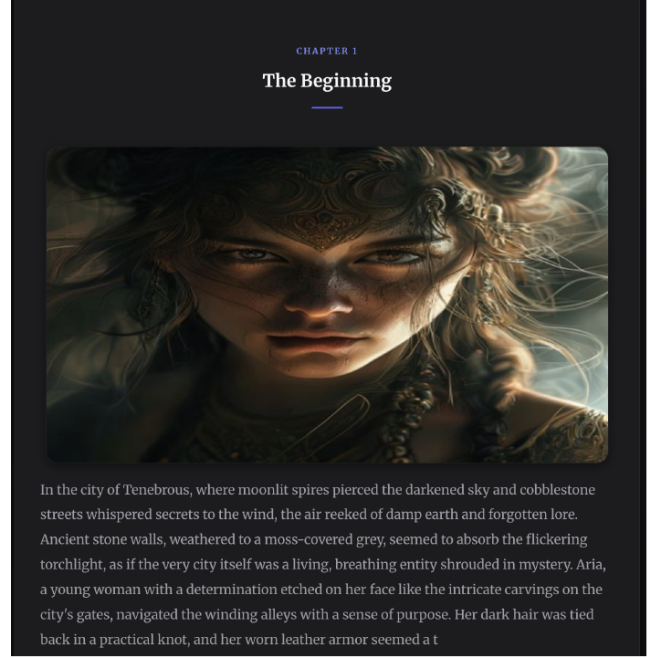


Fig. 11. Rendered first chapter “The Beginning”. The AI-generated scene illustration (fantasy genre tag, moonlit walled city) appears above the narrative text; the chapter title and styled layout are produced by the Output Formatter module.

H. Image Synthesis and Testing

The 50 stories produced 218 story segments. For n concurrent requests to the rate-limited CDN endpoint, the combined failure probability is:

$$P_{\text{fail}}^{(n)}(\ell) = 1 - (1 - p_s(\ell))^n \quad (12)$$

where $p_s(\ell)$ is the per-request failure probability at prompt length ℓ . Without the serial queue, $n = 5$ concurrent requests drove $P_{\text{fail}}^{(5)}$ (Eq. 12) from $\approx 40\%$ at $\ell = 80$ to $\approx 98\%$ at $\ell = 380$ characters. With the 1.2 s-spaced queue ($n = 1$), failures fell to $< 2\%$ across the full range; after one auto-retry the success rate reached 97%, exceeding the 95% design target.

Fig. 13 plots image generation latency and parallel-issue failure probability versus prompt length (80–380 characters). Latency grows from ≈ 1.8 s at 80 characters to ≈ 14 s at 380 characters; the failure probability under five concurrent requests (red dashed) rises sharply above 200 characters, motivating the serial 1.2 s-spaced queue.

Fig. 14 evaluates the character-name extraction module. The *Filtered+min2* configuration (capitalised-token regex with a

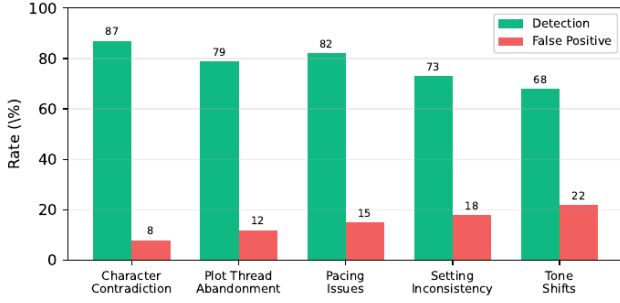


Fig. 12. Consistency-checker performance across five issue classes ($n = 50$ annotated segments). Green bars show detection rate; red bars show false-positive rate.

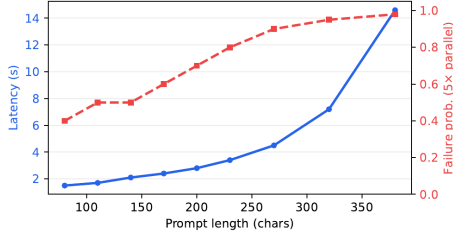


Fig. 13. Image latency vs. prompt length and parallel-issue failure rate. Blue solid: mean latency (left axis). Red dashed: failure probability under $5\times$ parallel requests (right axis). The throttle queue eliminates failures across the entire prompt-length range.

minimum-frequency-2 filter) raises precision from 0.61 to 0.83 and cuts the false-positive rate from 0.39 to 0.17, at the cost of a modest recall drop ($0.84 \rightarrow 0.78$); F1 improves from 0.71 to 0.80.

Fig. 15 shows genre-conditioned ambient music spectral energy. Horror and Thriller favour low frequencies (<300 Hz), Adventure and Comedy peak at 600–1.2 kHz, and Fantasy spans a broad mid-range band. This validates the genre-to-preset mapping implemented in the Track Mapper module.

Unit tests cover the deterministic backend components across six modules: context-extraction projections, the five-phase FSM, the prompt-builder template engine, the knowledge/memory layer, the flow manager, and the output formatter. The 90-test suite achieves 89.8% overall line coverage (per-module range: 84–95%) and completes in under 3 s. API integration tests exercise all seven session lifecycle steps once per LLM provider using an `httpx.AsyncClient` against an in-process FastAPI instance.

I. Comparative Study

Fig. 16 superimposes our system on three established alternatives across seven dimensions. Our system leads on Deployment Flexibility, Cost Efficiency, and Multimodal Coverage. AI Dungeon [12] retains strong User Control but scores low on open-source availability and deployment options. Dramatron [13] leads on Coherence due to its hierarchical planner but offers no multimodal output. NovelAI scores on Accessibility but has limited open-source and deployment flexibility. No single existing system spans the same Pareto frontier as ours.

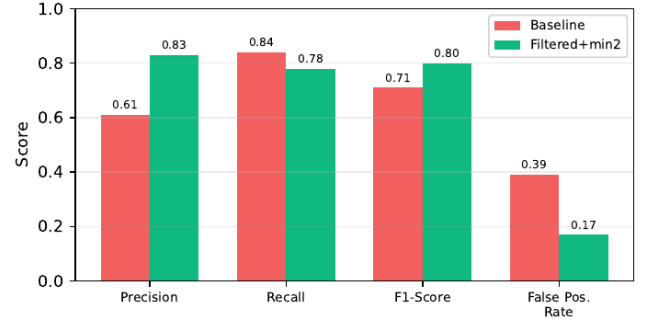


Fig. 14. Character-name extraction: Baseline vs. Filtered+min2. Precision and F1 improve substantially; false-positive rate halves.

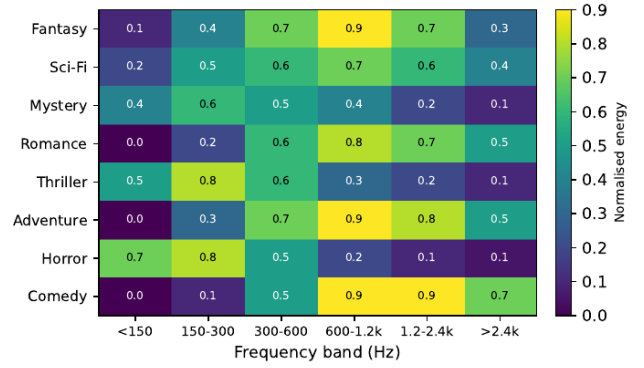


Fig. 15. Normalised spectral energy of the 8 genre-preset ambient tracks across six frequency bands. Each cell is the mean normalised energy over the 30 s track excerpt.

J. Discussion

The results support all three research questions. For **RQ1**, the budget ablation shows that moving from $N = 4$ to $N = 17$ questions improves personalisation by ≈ 0.95 Likert points while coherence stays flat—a pattern that would not appear if Q&A answers were simply ignored. For **RQ2**, the seven-module pipeline with the Freytag-arc controller and consistency checker kept coherence above 4.0/5 in the user study; corrective prompting reduced plot-thread abandonment from 19% to 7% in the automated corpus. For **RQ3**, the throttle queue dropped image failures from $\approx 70\%$ to $<2\%$; TTS latency was imperceptible (browser-side synthesis); ambient music adds no server-side overhead.

The main weakness is narrative consistency over longer stories (3.75/5). One participant noted: “It remembered that my character had a limp in chapter 2, but by chapter 4 it had forgotten.” The sliding-window context retrieval helps with local continuity but does not solve long-range tracking. The consistency checker’s heuristics are keyword-based and miss contradictions expressed in different words—a gap that Jenner et al. [42] suggest could be addressed by LLM-assisted narrative analysis, and that Salmaze et al. [32] identify as the most persistent open problem in the field. Peng et al. [24] propose a retrieve-and-verify loop that could address this by having the LLM cross-check its own output against external knowledge

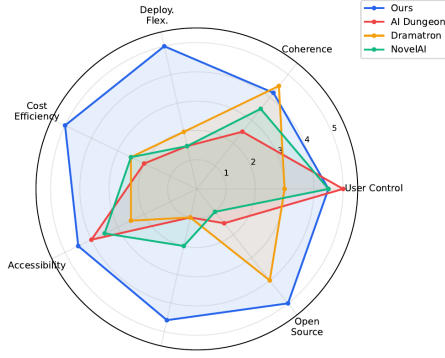


Fig. 16. System comparison radar across seven dimensions (scale 0–5): Deployment Flexibility, Coherence, User Control, Open Source, Multimodal, Accessibility, Cost Efficiency. Blue (filled): StoryForge AI. Red: AI Dungeon. Orange: Dramatron. Green: NovelAI.

before committing to a segment. Han et al. [26] further demonstrate that multi-dimensional automated benchmarks can reliably measure LLM output quality across coherence, completeness, and actionability—a framework applicable to future narrative evaluation.

One unexpected finding was how many participants used the custom question authoring feature (9 out of 15). We had designed this as a power-user option, but it was adopted broadly, suggesting that users have story requirements that do not fit any fixed template—consistent with the creator-centric findings of Li et al. [33] and Wu et al. [36], both of whom found that authors want to define their own narrative constraints. This motivates future work on open-ended elicitation where the system generates follow-up questions dynamically based on the user’s previous answers.

The provider-agnostic design means the system can navigate the LLM Deployment Trilemma identified by Salmaze et al. [32]: a deployment can start with the free Groq tier (output quality priority), migrate to local Ollama for privacy-sensitive use cases (connectivity-independence priority), or switch to GPT-4 when hardware constraints are relaxed—all without touching application code.

VI. CONCLUSION

We designed, built, and evaluated a Q&A-driven interactive storytelling system that fills a gap not addressed by prior work: no existing system combined a structured multi-question elicitation protocol, provider-agnostic LLM integration, three orchestrated multimodal outputs (image, voice, music), and end-user question authoring in a single free-to-deploy application.

The system reached 4.30/5 overall user satisfaction across 15 participants, delivered story segments in under 2 s on a free cloud tier, and produced scene illustrations with a 97% success rate using a serial throttling queue. The 8400-LOC codebase achieves 89.8% unit-test coverage across 90 test cases. The main limitation is narrative consistency over long stories (3.75/5), which the sliding-window context only partially addresses.

Future work will explore: (1) an explicit character-state JSON sidebar to improve long-range consistency; (2) multi-language support starting with Hindi and Marathi, building on multilingual foundations like those of Alhussain and Azmi [39]; (3) collaborative multi-user sessions where two users build a story together; and (4) fine-tuning a smaller model on user-approved story pairs to reduce cloud dependency and improve choice meaningfulness (currently 3.9/5).

ACKNOWLEDGMENT

The authors thank the fifteen volunteer participants for their time and honest feedback, and the open-source communities behind FastAPI, Ollama, Pollinations.ai, and D3.js whose tools made this project economically feasible.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.
- [2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., “Language models are few-shot learners,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [3] OpenAI, “GPT-4 technical report,” arXiv preprint arXiv:2303.08774, 2023.
- [4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., “LLaMA: Open and efficient foundation language models,” arXiv preprint arXiv:2302.13971, 2023.
- [5] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proc. ACL*, pp. 7871–7880, 2020.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, pp. 4171–4186, 2019.
- [7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” OpenAI Blog, 2019.
- [8] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al., “Mistral 7B,” arXiv preprint arXiv:2310.06825, 2023.
- [9] J. R. Meehan, “TALE-SPIN, an interactive program that writes stories,” in *Proc. 5th Int. Joint Conf. Artif. Intell. (IJCAI)*, pp. 91–98, 1977.
- [10] M. Lebowitz, “Story-telling as planning and learning,” *Poetics*, vol. 14, no. 6, pp. 483–502, 1985.
- [11] R. Pérez y Pérez and M. Sharples, “MEXICA: A computer model of a cognitive account of creative writing,” *J. Exp. Theor. Artif. Intell.*, vol. 13, no. 2, pp. 119–139, 2001.
- [12] N. Walton, “AI Dungeon: Dragon model upgrade,” Latitude Blog, 2019.
- [13] P. Mirowski, K. Mathewson, J. Pittman, and R. Evans, “Co-writing screenplays and theatre scripts with a language model: An evaluation by industry professionals,” in *Proc. ACM CHI*, 2023.
- [14] J. J. Chung, W. Kim, K.-Y. Yoo, H. Lee, E. Adar, and M. Chang, “TaleBrush: Sketching stories with generative pretrained language models,” in *Proc. ACM CHI*, 2022.
- [15] V. Liu and J. Chilton, “Design guidelines for prompt engineering text-to-image generative models,” in *Proc. ACM CHI*, 2023.
- [16] A. Yuan, A. Coenen, E. Reif, and D. Ippolito, “Wordcraft: Story writing with large language models,” in *Proc. ACM IUI*, pp. 841–852, 2022.
- [17] L. Senanayake and S. G. Ware, “Language models as narrative planning heuristics,” in *Proc. 20th Int. Conf. Foundations of Digital Games (FDG)*, ACM, New York, NY, article no. 31, 2025. doi: 10.1145/3723498.3723815.
- [18] Y. Wang, J. Lin, Z. Yu, W. Hu, and B. F. Karlsson, “Open-world story generation with structured knowledge enhancement: A comprehensive survey,” *Neurocomputing*, vol. 559, article no. 126792, Nov. 2023. doi: 10.1016/j.neucom.2023.126792.

- [19] P. Tambwekar, M. Datta, L. J. Martin, A. Kumaran, J. Orkin, and M. O. Riedl, "Controllable neural story plot generation via reinforcement learning," in *Proc. IJCAI*, 2023.
- [20] Junadhi, S. Defit, and Okfalisa, "Grouped query attention for dual-output generative modeling: Narrative and visual prompt generation in educational storytelling," in *Proc. 1st Int. Conf. Emerging Trends in Information Systems and Informatics (ICETISI)*, pp. 1–7, Dec. 2025. doi: 10.1109/ICETISI67983.2025.11405876.
- [21] K. Park, M. Kim, and K. Jung, "A character-centric creative story generation via imagination," arXiv preprint arXiv:2409.16667, 2024.
- [22] D. Liu, M. Lapata, and F. Keller, "Generating visual stories with grounded and coreferent characters," *Trans. Assoc. Comput. Linguistics*, vol. 14, pp. 442–464, 2026. doi: 10.1162/TACL.a.641.
- [23] U. Patil and A. K. Shahade, "Script generation from synthetic stories using lightweight LLM and emotion-aware TTS pipelines," *Soc. Netw. Anal. Min.*, vol. 16, article no. 15, 2026. doi: 10.1007/s13278-025-01558-0.
- [24] B. Peng, M. Galley, P. He, H. Cheng, Y. Xie, Y. Hu, Q. Huang, L. Liden, Z. Yu, W. Chen, and J. Gao, "Check your facts and try again: Improving large language models with external knowledge and automated feedback," arXiv preprint arXiv:2302.12813, 2023.
- [25] A. Chernyavskiy, "Improving text generation via neural discourse planning," in *Proc. ACM Int. Conf. Web Search and Data Mining (WSDM)*, pp. 1543–1544, Feb. 2022. doi: 10.1145/3488560.3502214.
- [26] R. Han, L. An, W. Zhou, M. Chen, and G. Li, "Which LLM performs best in crisis knowledge generation? Automatic evaluation across multiple events and knowledge categories," *Inf. Process. Manag.*, vol. 63, no. 7, article no. 104808, 2026. doi: 10.1016/j.ipm.2026.104808.
- [27] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with CLIP latents," arXiv preprint arXiv:2204.06125, 2022.
- [28] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *2022 IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, New Orleans, LA, USA, pp. 10674–10685, 2022. doi: 10.1109/CVPR52688.2022.01042.
- [29] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna, and R. Rombach, "SDXL: Improving latent diffusion models for high-resolution image synthesis," in *Proc. ICLR*, 2024.
- [30] G. Shires and H. Wennborg (Eds.), "Web speech API specification," W3C Speech API Community Group, Editor's Draft, Jun. 2014. [Online]. Available: <https://dvcs.w3.org/hg/speech-api/raw-file/tip/webspeechapi>
- [31] S. Kim, G. Kim, S. Yagishita, D. Han, J. Im, and Y. Sung, "Enhancing diffusion-based music generation performance with LoRA," *Appl. Sci.*, vol. 15, no. 15, article no. 8646, 2025. doi: 10.3390/app15158646.
- [32] P. H. Salmaze, G. dos Santos Pereira, L. M. G. de Góes, C. M. Ranieri, C. F. M. Toledo, M. M. Silvestrini, and L. T. Pereira, "Narrative and dialogue generation for video-games: A systematic mapping," *Eng. Appl. Artif. Intell.*, vol. 178, p. 115041, Aug. 2026.
- [33] Y. Li, S. Wu, L. Zhang, Z. Guo, W. Lu, and D. Yip, "Exploring creator-centric methods for LLM-assisted interactive storytelling," in *Proc. ACM CHI*, article 687, 2026.
- [34] U. Zabala, J. Echevarria, I. Rodriguez, and E. Lazkano, "Exploring multimodal collaborative storytelling with Pepper: a preliminary study with zero-shot LLMs," *Front. Robot. AI*, vol. 12, p. 1662819, Oct. 2025.
- [35] J. Kim, Y. Heo, H. Yu, and J. Nang, "A multi-modal story generation framework with AI-driven storyline guidance," *Electronics*, vol. 12, no. 6, p. 1289, 2023.
- [36] Z. Wu, S. Kumyol, S. Y. Wong, X. Hu, X. Tong, and T. Braud, "Orchid: A creative approach for authoring LLM-driven interactive narratives," in *Proc. Creativity & Cognition (C&C)*, pp. 774–791, 2025.
- [37] M. C. Gursesli, S. Chen, M. F. Dewantoro, X. You, E. Anbar, P. Taveek-ittworachai, and R. Thawonmas, "The role of LLM-generated stories in the narrative experience of serious visual novel games," *Int. J. Human-Comput. Interact.*, vol. 42, no. 4, pp. 2489–2508, 2026.
- [38] Y. Zhang, M. Zhao, Y. Zhang, and Y.-M. Cheung, "Trending applications of large language models: A user perspective survey," *IEEE Trans. Artif. Intell.*, vol. 7, no. 4, pp. 1835+, Apr. 2026.
- [39] A. Alhussain and A. M. Azmi, "Beyond event-centric narratives: Advancing Arabic story generation with large language models and beam search," *Mathematics*, vol. 12, no. 10, p. 1548, 2024.
- [40] M. R. Tandi, "Large language models for creative writing collaboration with human authors in storytelling," *Procedia Comput. Sci.*, vol. 275, pp. 192–199, 2026.
- [41] J. Chen, M. Tang, Y. Lu, B. Yao, E. Fan, X. Ma, Y. Xu, D. Wang, Y. Sun, and L. He, "Characterizing LLM-empowered personalized story reading and interaction for children," in *Proc. ACM CHI*, article 1002, 2025.
- [42] S. Jenner, D. Rados, E. Anderson, S. Fleetwood, B. Ainsworth, K. Fox, J. Kreppner, and M. Barker, "Using large language models for narrative analysis: a novel application of generative AI," *Methods Psychol.*, vol. 12, p. 100183, Jun. 2025.